

CAPX

Cyber Analysis & Protection eXplorer

Case Study, Product Specification & Complete Software Structure

Product Type: Windows endpoint analysis, system-health, and internal defense software

Document Purpose: Establish the functional scope, architecture, case study, and development foundation before implementation.

1. Project Overview

1.1 Product Identity

CAPX — Cyber Analysis & Protection eXplorer

1.2 Primary Purpose

CAPX analyzes a Windows computer to identify system-performance problems, resource-heavy processes, suspicious process behavior, network activity, persistence indicators, system exposure, and behavioral anomalies. It then provides risk assessment, findings, explanations, recommendations, reports, and safe user-approved optimization actions.

2. Case Study

2.1 Background

Modern Windows computers continuously run numerous processes, background services, startup applications, network connections, and scheduled activities. When a computer becomes slow or behaves unexpectedly, users may have difficulty determining whether the cause is resource consumption, unnecessary background applications, startup programs, network activity, abnormal processes, persistence mechanisms, or potentially suspicious behavior.

2.2 Problem Statement

Users need a single tool that can examine the condition of a Windows system and provide understandable information about both performance and potential security concerns. Basic monitoring utilities may expose CPU, RAM, processes, or network connections individually, while users still need to interpret the information manually.

2.3 Proposed System

CAPX provides a unified local analysis platform that collects system information, analyzes processes, network activity, startup mechanisms, and system health, correlates relevant indicators, calculates a risk assessment, and provides understandable recommendations.

3. Core Product Modes

3.1 Inspect

`capx inspect` — Full system and security analysis.

3.2 Monitor

`capx monitor` — Continuous monitoring of system and security indicators.

3.3 Optimize

`capx optimize` — Guided performance optimization and cleanup.

4. Objectives

4.1 Objectives

- Analyze Windows system health.
- Identify resource-heavy processes.
- Monitor relevant network activity.

- Identify persistence indicators.
- Detect behavioral anomalies.
- Correlate multiple security indicators.
- Provide explainable risk assessments.
- Generate actionable recommendations.
- Provide continuous monitoring.
- Provide safe, user-approved optimization.
- Generate system/security reports.
- Package the application as downloadable Windows software.

5. Scope

5.1 Included

Windows system analysis; process analysis; network analysis; startup analysis; system-health analysis; anomaly detection; risk assessment; recommendations; monitoring; reporting; guided optimization; local Flask-based interface; Windows executable packaging.

5.2 Initial Boundaries

CAPX is not initially intended to provide enterprise EDR functionality, guaranteed malware detection, automatic removal of unknown malware, cloud threat intelligence, or replacement of commercial antivirus software.

6. Proposed System Workflow

6.1 Overall Flow

WINDOWS SYSTEM ↓ DATA COLLECTION ↓ PROCESS / NETWORK / STARTUP / SYSTEM HEALTH ↓ ANALYSIS ENGINE ↓ ANOMALY ENGINE ↓ RISK ASSESSMENT ↓ FINDINGS + RECOMMENDATIONS + REPORTS ↓ USER-APPROVED OPTIMIZATION

7. Complete Software Architecture

7.1 Target Directory Structure

```
CAPX/
├── app.py
├── capx.py
├── cli.py
├── config.py
├── requirements.txt
├── README.md
├── LICENSE
├── core/
│   ├── collector/
│   │   ├── system_collector.py
│   │   ├── process_collector.py
│   │   ├── network_collector.py
│   │   └── startup_collector.py
│   ├── service_collector.py
│   └── analysis/
│       ├── process_analyzer.py
│       ├── network_analyzer.py
│       ├── startup_analyzer.py
│       ├── health_analyzer.py
│       ├── exposure_analyzer.py
│       └── detection/
│           ├── anomaly_engine.py
│           ├── behavior_engine.py
│           ├── indicators.py
│           ├── signatures.py
│           └── risk/
│               ├── scorer.py
│               ├── risk_levels.py
│               ├── confidence.py
│               ├── optimization/
│               │   ├── optimizer.py
│               │   ├── cleanup.py
│               │   ├── actions.py
│               │   ├── recommendations/
│               │   └── engine.py
│               ├── api/
│               ├── system_routes.py
│               ├── process_routes.py
│               ├── network_routes.py
│               ├── security_routes.py
│               ├── monitoring_routes.py
│               ├── optimization_routes.py
│               ├── permissions.py
│               ├── validation.py
│               ├── audit_log.py
│               ├── safe_actions.py
│               ├── monitoring/
│               │   ├── monitor.py
│               │   ├── scheduler.py
│               │   └── event_manager.py
│               ├── reporting/
│               │   ├── report.py
│               │   └── exporter.py
```

templates.py ■ ■■■ data/ ■ ■■■ database.py ■ ■■■ models.py ■ ■■■ templates/ ■■■
static/ ■■■ reports/ ■■■ logs/ ■■■ tests/ ■■■ docs/

8. Core Modules

8.1 System Collector

Collects CPU, RAM, disk, battery, uptime, and system information.

8.2 Process Collector

Collects PID, process name, executable path, CPU usage, memory usage, parent process, user, and creation time where available.

8.3 Network Collector

Collects active connections, local and remote addresses, connection status, and listening ports.

8.4 Startup Collector

Checks relevant Windows startup and autorun mechanisms.

8.5 Analysis Layer

Interprets collected facts without automatically declaring a process malicious.

8.6 Detection Engine

Correlates indicators and identifies behavioral anomalies.

8.7 Risk Engine

Produces a structured risk score and confidence information.

8.8 Recommendation Engine

Converts findings into understandable, actionable recommendations.

8.9 Reporting Layer

Produces readable reports and exportable results.

9. Risk Assessment

9.1 Initial Risk Bands

0–25 SAFE

26–50 MONITOR

51–75 WARNING

76–100 CRITICAL

9.2 Important Design Rule

The initial score should be described as a CAPX risk score, not as a literal malware probability, unless the scoring system is later validated and calibrated statistically.

10. Optimization and Safety

10.1 Safe Workflow

ANALYZE ↓ IDENTIFY CANDIDATE ACTION ↓ SHOW WHAT WILL CHANGE ↓ USER CONFIRMATION ↓
SAFETY VALIDATION ↓ EXECUTE ↓ VERIFY ↓ AUDIT LOG

10.2 Safety Principle

CAPX should not automatically terminate unknown processes, delete system-critical files, or classify every high-resource process as a security threat. High CPU usage alone is not sufficient evidence of malicious activity.

11. Required Diagrams

11.1 Design Documentation

- System Architecture Diagram
- Use Case Diagram
- Context Diagram
- DFD Level 0
- DFD Level 1
- Activity Diagram — Inspect
- Activity Diagram — Monitor
- Activity Diagram — Optimize
- Sequence Diagram
- Threat/Security Model
- ER/Data Model Diagram

12. Development Principle

12.1 Build Standard

Requirements → Architecture → Diagrams → Security Model → Implementation → Testing → Hardening → Packaging → Release

12.2 Product Standard

CAPX should be developed as serious security software: correctness, safety, maintainability, explainability, reliability, performance, testing, and secure defaults take priority over adding flashy features.

13. CAPX Product Architecture at a Glance

CAPX ██████████ ▼ ▼ ANALYZE DEFEND OPTIMIZE ■ ■ System Health
Anomaly Safe Actions Processes Monitoring Cleanup Network Risk Engine Performance Startup Recommendations
■ ■ ██████████ ▼ REPORTING ■ ▼ USER

This document establishes the baseline specification. Individual modules should be implemented only after their responsibilities, inputs, outputs, permissions, failure behavior, and testing requirements are defined.